

ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej

Web System Programming

W04 & W05 – CSS3, a brief overview of
the capabilities

Syllabus

- CSS3 – overview of the basic capabilities
 - Style format
 - Style sheets
 - Color recording
 - Styles for fonts
 - Line `<span>` and block `<div>` elements
 - Basic selectors
 - Complex selectors
 - Style hierarchy
 - Positioning of elements
 - Displaying items
 - An example of a pseudo-class and its use
 - Viewport
 - Media queries
 - Responsibility
 - Overview of other CSS3 capabilities
 - Emmet in VS Code

Introduction to styles

- CSS is the language we use to style the web page (document styling).
- CSS describes **how to display** HTML elements.
- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or other media
- CSS saves a lot of work. It can control the layout of multiple websites simultaneously
- External style sheets are stored in CSS files

Style format

- Each style element consists of a pair:
 - CSS3 style properties
 - The values of this property (sometimes called a style attribute, or even an attribute)
- These values are **separated** by a **colon**
- Successive **pairs** of styles are **separated** by a **semicolon**
- There may also be a semicolon after the last pair, which makes it easier to edit the style.

```
Property1:value1; Property2:value2; ...; PropertyN:valueN
```

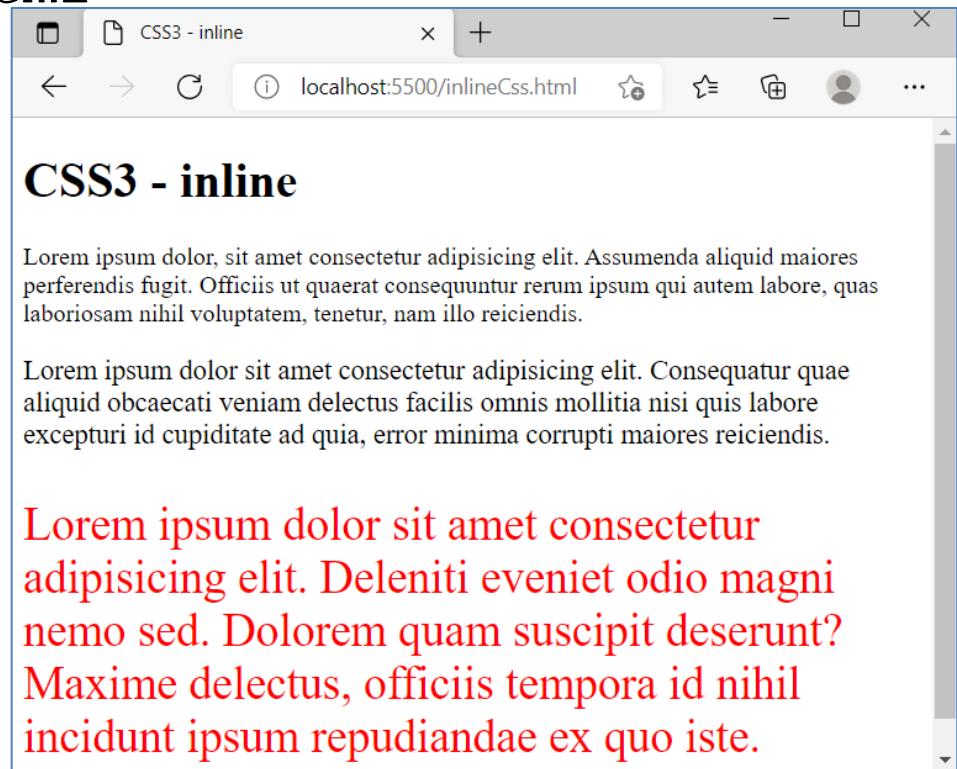
```
Property1:value1;  
Property2:value2;  
...;  
PropertyN:valueN
```

```
Property1:value1;  
Property2:value2;  
...;  
PropertyN:valueN;
```

Inline styles

- The visual presentation (colors, fonts, position, behavior) of HTML elements can be changed.
- Part of the attributes of element or the general attribute **style** can be used for this.
 - this is **inline** use.
 - Example: `inlineCss.html`

```
<title>CSS3 - inline</title>
</head>
<body>
  <h1>CSS3 - inline</h1>
  <p>
    Lorem ipsum dolor, sit ...
  </p>
  <p style="font-size:large">
    Lorem ipsum dolor sit ...
  </p>
  <p style="font-size: 30px; color:red">
    Lorem ipsum dolor sit ...
  </p>
</body>
</html>
```



CSS3 stylesheet format

- Style **sheet**
- It is a **set of rules** that follow one another, separated by whitespace
- Each **rule** is a **pair**:
 - CSS3 selector
 - Style described as in the *inline* method but enclosed in **curly braces**
- one of the **simpler selectors** is the name of the html element (**p**, **h1**, etc.)

```
Selector1 {  
    property1_1: value1_1;  
    ...  
}  
Selector2 {  
    property2_1: value2_1;  
    ...  
}  
...
```

```
p {  
    font-size: 10px;  
    color:red;  
}  
h1 {  
    color:blue  
}
```

Embedded CSS3 style sheets (in `<head>` element)

- Instead of using the inline method, you can (should) collect the styles into a certain set of rules - a stylesheet.
- Style sheet can be specified in the `<head>` element directly (as **embedded** style sheets)
 - Using the `<style>` element
 - Example: `inHeadCss.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    p {
      font-size: 10px;
      color:red
    }
    h1 {
      color:blue
    }
  </style>
  <title>CSS3 - in Head</title>
</head>
<body>
  <h1>CSS3 - in Head</h1>
  <p>
    Lorem ipsum dolor, sit ...
  </p>
  <p>
    Lorem ipsum dolor sit ...
  </p>
</body>
</html>
```

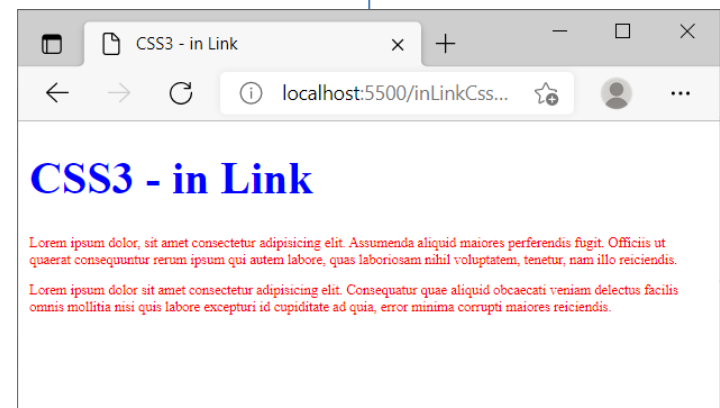


External CSS3 style sheets

- The style sheet can also be in a different file.
- The `<head>` element then contains information about the stylesheet using an empty `<link>` element with the **rel** = "stylesheet" attribute and the **href** attribute with a **link to this file**.
- Example:
 - inLinkCss.html
 - inLink.css
- This form (**external** style sheets) is most **desirable**.
- You can **comment** between `/ *` and `* /` in embedded and external style sheets.
- You can insert **as many** external style sheets **as you want** in an HTML document.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="inLink.css">
  <title>CSS3 - in Link</title>
</head>
<body>
  <h1>CSS3 - in Link</h1>
  <p>
    Lorem ipsum dolor, sit amet ...
  </p>
  <p>
    Lorem ipsum dolor sit amet ...
  </p>
</body>
</html>
```

```
p {
  font-size: 10px;
  color: red;
}
h1 {
  color: blue
```

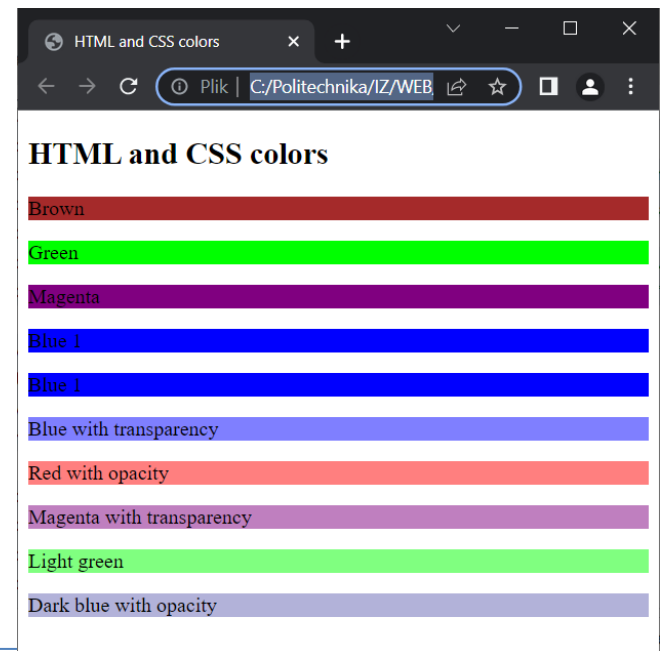


Basic properties in style sheets - color

- Colors:
 - **color** property - font color
 - **background-color** property - background color
- Color can be expressed as:
 - The symbolic name of the color, e.g. **red**, **lightgreen**, **aqua**
 - Hexadecimal code in the format "**#RRGGBB**" eg **#AB081F**
 - Hexadecimal code in the format "**#RGB**" eg **#A18**, which means **#AA1188**
 - The tuples „**rgb** (**R**, **G**, **B**) " - each with a value in the range **0-255**.
 - The tuples „**rgb** (**R%**, **G%**, **B%**) " - each value in the range **0-100**.
 - The tuples "**rgba** (**R**, **G**, **B**, **alpha**) " - **alpha** is a number between **0.0** (fully transparent) and **1.0** (not at all transparent)
 - The tuples "**hsl** (**H**, **S%**, **L%**) " - **H** - color **0-360** from the color wheel, **S** - saturation **0-100**, **L** - brightness **0-100**.
 - The tuples "**hsla** (**H**, **S%**, **L%**, **alpha**) " - **alpha** is a number ranging from **0.0** (fully transparent) to **1.0** (not at all transparent)

Ways of defining colors in CSS

```
<!DOCTYPE html>
<html><head>
<style>
#p1 {background-color:brown;} /* Color Name - 140 color names are predefined */
#p2 {background-color:rgb(0,255,0);} /* rgb(red, green, blue). Each parameter (red, green, and blue) defines the intensity of the color and can be an integer between 0 and 255 */
#p3 {background-color:rgb(50%,0%,50%);} /* (red, green, and blue) defines the intensity of the color and can be a percentage value (from 0% to 100%). */
#p4 {background-color:#0000FF;} /* #RRGGBB, where the RR (red), GG (green) and BB (blue) hexadecimal */
#p5 {background-color:#00F;} /* =#0000FF */
#p6 {background-color:#0000FF80;} /* Hexadecimal Colors With Transparency. To add transparency, add two additional digits between 00 and FF. */
#p7 {background-color:rgba(255,0,0,0.5);} /* rgba(red, green, blue, alpha) - The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (fully opaque). */
#p8 {background-color:rgb(50%,0%,50%,50%);}
#p9 {background-color:hsl(120, 100%, 75%);} /* hsl(hue, saturation, lightness) - Hue is a degree on the color wheel (from 0 to 360) - 0 (or 360) is red, 120 is green, 240 is blue. Saturation is a percentage value; 0% means a shade of gray and 100% is the full color. Lightness is also a percentage; 0% is black, 100% is white.*/
#p10 {background-color:hsla(240, 100%, 25%, 0.3);} /*hsla(hue, saturation, lightness, alpha) The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (fully opaque).*/
</style>
<title>HTML and CSS colors</title>
</head><body>
<h2>HTML and CSS colors</h2>
<p id="p1">Brown</p>
<p id="p2">Green</p>
<p id="p3">Magenta</p>
<p id="p4">Blue 1</p>
<p id="p5">Blue 1</p>
<p id="p6">Blue with transparency</p>
<p id="p7">Red with opacity</p>
<p id="p8">Magenta with transparency</p>
<p id="p9">Light green</p>
<p id="p10">Dark blue with opacity</p>
</body>
</html>
```



Basic properties in style sheets - font

- The **font-weight** property - font thickness (weight):
 - Predefined absolute symbolic values: `bold`, `normal`,
 - Numeric values are `0-1000`, where `400` is `normal` and `700` is `bold`.
 - Relative values (style inheritance): `bolder`, `lighter`
- The **font-size** property - font size:
 - Predefined absolute symbolic values: `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large`.
 - Symbolic relative values (style Inheritance): `smaller`, `larger`
 - Percentage: `x%` of the value of the **current** font
 - Expressed in the values of the acceptable **units**

Basic Properties in Style Sheets - Font Family

- The **font-family** property - a sequence of font types that the browser will try to use if it doesn't find the first one.
 - Consecutive types of fonts separated by a comma.
- They can be very specific **types**, or they can be **font families** with a certain property
 - It is best to give the font family **at the end** of the sequence.
- Font families and specific fonts:
 - **Serif** - with ornaments at the ends
 - Times New Roman, Georgia
 - **Sans-serif** - without ornaments
 - Arial, Verdana, futura
 - **Monospace** - Fixed-width characters
 - Courier new, fixedsys
 - **Cursive** - inclined
 - script
 - **Fantasy** - fancy
 - Critter, Copperplate, Papyrus
- Note: On computer screens, sans-serif fonts are considered easier to read than serif fonts.
- Source: https://www.w3schools.com/css/css_font.asp



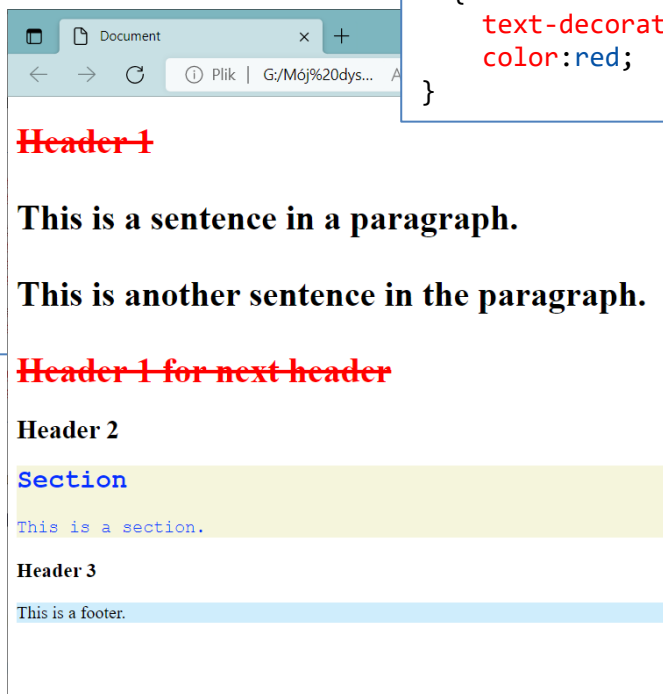
Font decoration

- The **text-decoration** property - imposes decorations in the text and is a shortcut to setting four other properties in one property (in order):
 - **text-decoration-line** (required)
 - **text-decoration-color**
 - **text-decoration-style**
 - **text-decoration-thickness**
- Values for **text-decoration-line**:
 - **underline** – underline
 - Note: It is not recommended to underline **non-link text** as this will often **confuse** the reader.
 - **overline** - a line above the text
 - **line-through** - strikethrough in the middle of the text
 - **none** - no line (default)
- The values for **text-decoration-color** are simply the decoration color.
- The **text-decoration-style** values are:
 - **solid** - solid line (default)
 - **double** - double line
 - **dotted** - dotted line
 - **dashed** - dashed line
 - **wavy** - wavy line
- The **text-decoration-thickness** property defines the thickness of the decoration lines, e.g. auto, 5px, 50%.
- Other possible values for all of the above properties are:
 - **initial** - return to default settings
 - **inherit** - inheriting values from the parent.
- Some of these values can be combined, for example:
 - **text-decoration**: **underline overline wavy red 50%**;
 - **text-decoration**: **underline line-through dashed blue 5px**;

Example – fonts (fonts.html, fonts.css)

```
<link rel="stylesheet" href="fonts.css">
<title>Document</title>
</head>
<body>
  <h1> Header 1</h1>
  <p>
    This is a sentence in a paragraph.
  </p>
  <p>
    This is another sentence in the paragraph.
  </p>
  <h1> Header 1 for next header</h1>
  <h2> Header 2</h2>
  <section>
    <h2>Section</h2>
    This is a section.
  </section>
  <h3> Header 3</h3>
  <footer>
    This is a footer.
  </footer>
</body>
</html>
```

```
p{
  font-size: 200%;
  font-weight: 800;
}
section{
  font-family: 'Courier New', Courier, monospace;
  background-color: beige;
  color:#0430FF;
}
footer{
  background-color:hsl(200,90%,90%)
}
h1{
  text-decoration: line-through;
  color:red;
}
```



Text transformation

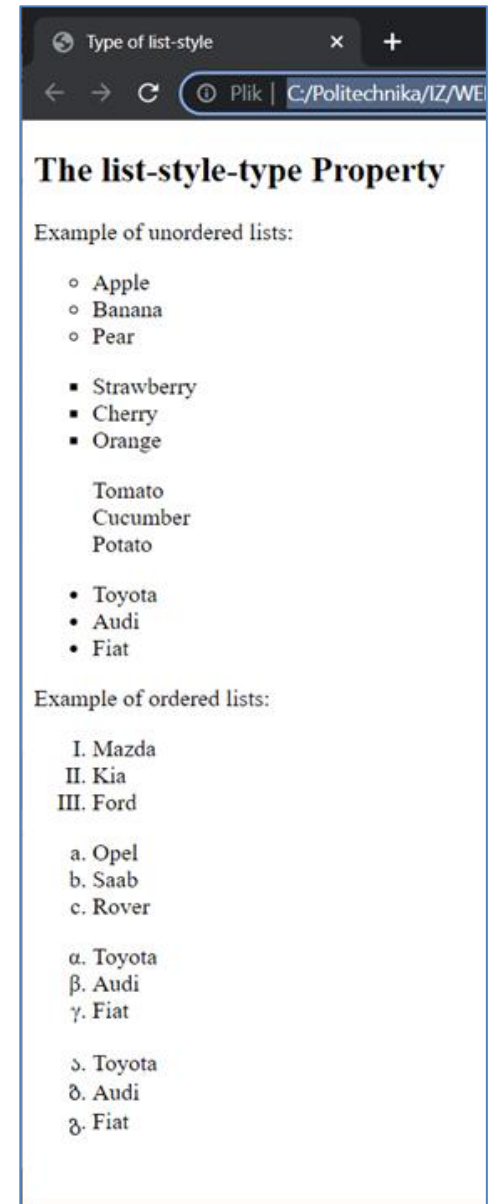
- Properties:

- **text-transform** - can be used to change case of letters in text: `none` | `capitalize` | `uppercase` | `lowercase` | `initial` | `inherit`;
- **text-indent** - specify the amount of indentation for the first line of text.
- **letter-spacing** - space between characters in the text.
- **line-height** - space between lines.
- **word-spacing** - space between words in the text.
- **white-space** - how to handle whitespace inside an element: `normal` | `nowrap` | `pre` | `pre-line` | `pre-wrap` | `initial` | `inherit`;
- **text-align**: `left`, `right aligned`, `center`, `justify` - horizontal text alignment
- **text-align-last** - how to align the last line of text
- **direction**: `ltr`, `rtl`, `unicode-bidi` - specifies the text direction of the item
- **vertical-align** - the vertical alignment of the element

Formatting bullets

```
<!DOCTYPE html>
<html>
  <!--file: type_list.html-->
<head>
<style>
ul.a { list-style-type: circle;}
ul.b { list-style-type: square;}
ul.c { list-style-type: none;}
ul.d { list-style-type: disc;}
ol.aa { list-style-type: upper-roman;}
ol.bb { list-style-type: lower-alpha;}
ol.cc { list-style-type: lower-greek;}
ol.dd { list-style-type: georgian;}
</style>
<title>Type of list-style </title>
</head>
<body>
<h2>The list-style-type Property</h2>
<p>Example of unordered lists:</p>
<ul class="a">
  <li>Apple</li>
  <li>Banana</li>
  <li>Pear</li>
</ul>
<ul class="b">
  <li>Strawberry</li>
  <li>Cherry</li>
  <li>Orange</li>
</ul>
```

```
<ul class="c">
  <li>Tomato</li>
  <li>Cucumber</li>
  <li>Potato</li>
</ul><ul class="d">
  <li>Toyota</li>
  <li>Audi</li>
  <li>Fiat</li>
</ul>
<p>Example of ordered lists:</p>
<ol class="aa">
  <li>Mazda</li>
  <li>Kia</li>
  <li>Ford</li>
</ol>
<ol class="bb">
  <li>Opel</li>
  <li>Saab</li>
  <li>Rover</li>
</ol>
<ol class="cc">
  <li>Toyota</li>
  <li>Audi</li>
  <li>Fiat</li>
</ol>
<ol class="dd">
  <li>Toyota</li>
  <li>Audi</li>
  <li>Fiat</li>
</ol>
</body>
</html>
```



Line elements ****, block **<div>**, and classes

- Within HTML there are **line** elements like ****, **<sup>**, ****, **** etc.
 - Have a meaning (semantics)
- There is one **** *line element* with **no semantics**
- Within HTML there are **block** elements like **<h1>**, **<p>**, **<section>** etc.
 - Have a meaning (semantics)
- There is one **<div>** *block element* with **no semantics**.
- Practically any HTML element can be given a **class** through the **class** attribute.
 - It is just the class ID
 - **It is NOT** a class in the programming sense, rather it is semantically **a set to which a given element belongs**
- An element **can belong to many classes**, then (in quotation marks) **separate** class names with a **space**
- By adding a class to the **** and **<div>** elements, we give them, in a sense, **meaning**.
- The **class name** is used very often in CSS3 **selectors**

Basic CSS3 selectors

- **HTML element name:**
 - No special preceding character
- **The name of the class**
 - Preceded by a period '.'
- **ID**
 - Preceded by the hash sign '#'
- **The name of a pseudo-class, a pseudo-element**
 - Preceded by a colon ':'
 - Detailed explanation on later slides
 - In the example, it means the event of hovering the mouse pointer over an element (here, the entire document).
- **All elements:**
 - Multiplication sign: '*'
- **Examples:**
 - `simpleSelectors.html`
 - `simpleSelectors.css`

```
p {
    font-size: 15px;
}
.important{
    color:red;
}
#bottom{
    font-weight: 800;
}
: hover{
    background-color: yellow;
}
```

```
<link rel="stylesheet" href="simpleSelectors.css">
<title>CSS3 - simple selectors</title>
</head>
<body>
  <h1 class="important">CSS3 - simple selectors</h1>
  <p class="important">
    Lorem ipsum dolor, sit ...
  </p>
  <p>
    Lorem ipsum dolor sit ...
  </p>
  <p id="bottom">
    Lorem ipsum dolor, sit ...
  </p>
</body>
</html>
```



CSS3 - simple selectors

Lorem ipsum dolor, sit amet consectetur adipisicing elit. Assumenda aliquid maiores perferendis fugit. Officiis ut queraat consequuntur rerum ipsum qui autem labore, quas laboriosam nihil voluptatem, tenetur, nam illo reiciendis.

Lorem ipsum dolor sit amet consectetur adipisicing elit. Consequatur quae aliquid obcaecati veniam delectus facilis omnis mollitia nisi quis labore excepturi id cupiditate ad quia, error minima corrupti maiores reiciendis.

Lorem ipsum dolor, sit amet consectetur adipisicing elit. Magnam hic rerum sequi reiciendis at repellendus blanditiis sed eum, ipsam ipsa suscipit quod numquam ab obcaecati, est cupiditate sunt possimus? At!

CSS3 - combining selectors

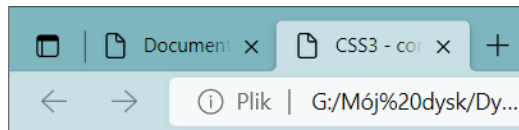
- The strength of CSS3 is that a selector can be a concatenation of other selectors. This connection can be done in many ways:
 - **With a comma** ",", which means that the style applies **to each of them separately**:
 - For example **p, h1, strong** {...} means that the style written in braces applies to the elements **<p>**, **<h1>** and ****
 - By writing **without gaps**, which means that **all selectors** must be **satisfied**:
 - For example, **p.important.hard** {...} means that the style written in braces refers to **<p>** elements that **must belong to two classes** simultaneously: **important** and **hard**.
 - **Via a space** eg "**sel1 sel2**" which means it refers to the **sel2** elements that are inside the **sel1** elements (**anywhere inside**).
 - **Via greater than sign** ">", such as "**sel1>sel2**", which means that the **sel2** element is **directly inside** the **sel1** element.

An example of complex selectors – complexSelectors (.html/.css)

```
section p{
    color:green;
}
section > p{
    background-color: yellow;
}

p.important.hard{
    font-size: larger;
}
```

```
<link rel="stylesheet" href="complexSelectors.css">
<title>CSS3 - complex selectors</title>
</head>
<body>
  <h1 class="important">CSS3 - complex selectors</h1>
  <section class="outer">
    <h2>Important hard</h2>
    <p class="important hard">
      Lorem ipsum dolor sit amet ...
    </p>
    <h2>Important medium</h2>
    <div class="inner">
      <p class="important medium">
        Lorem ipsum dolor sit amet ...
      </p>
    </div>
  </section>
</body>
</html>
```



CSS3 - complex selectors

Important hard

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Consectetur esse voluptatibus sequi repellendus alias libero a vitae ad reiciendis reprehenderit?

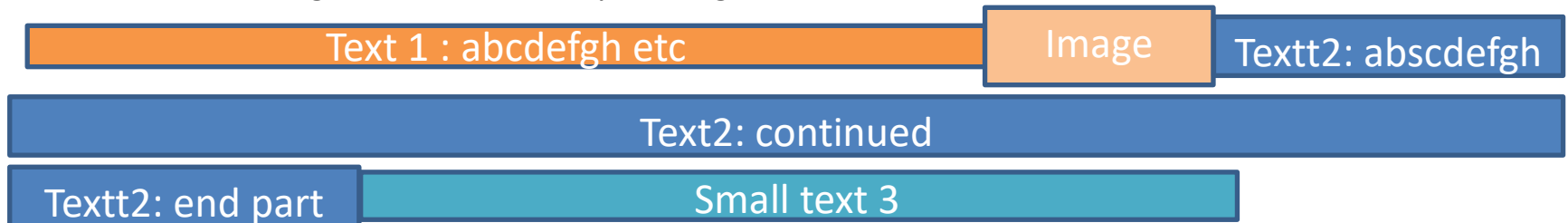
Important medium

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Harum iure illo facere tempora enim quas natus fugit aperiam, voluptatibus cum!

Row and block elements

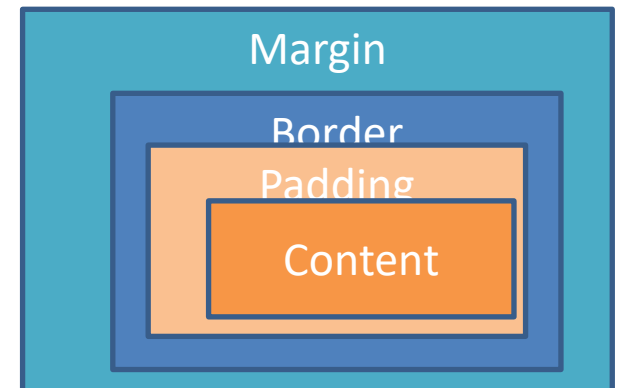
By default (**position** = `static`), subsequent HTML elements go to the rendering queue and:

- Row elements are placed sequentially with lines.
 - The line height is determined by the highest element of the line.



- Block elements are always on a new line (occupy one entire line).

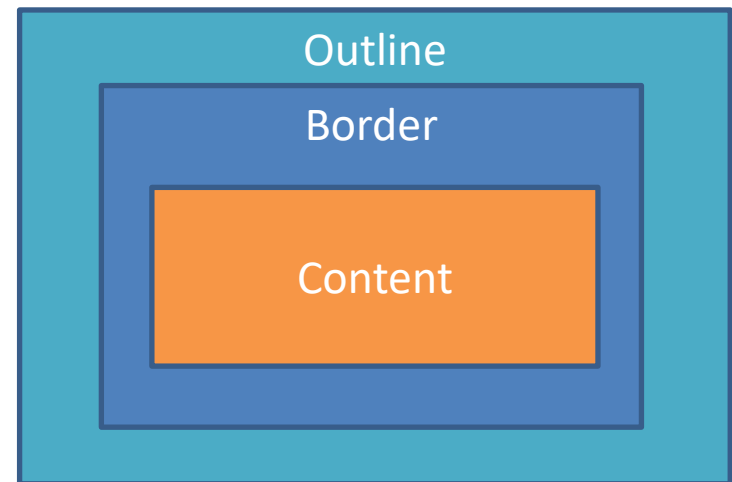
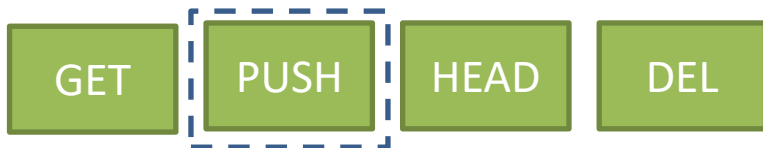
- They have **virtual fields** around them:
 - **Padding** - spacing from content to frame
 - **Border** - frame
 - **Margin** - space from the surrounding frame to the others
 - Different operation depending on the location of the element
- You can add colors, background images etc:
 - Property **background-image**
 - Property **background-position**
 - Property **background-repeat**
 - Property **background-attachment**:
 - Property **text-indent**
 - Property **font-style**



- The **widths** of virtual fields (in order: top, bottom, left, right) can be set together or independently.

Outline

- An outline is a line drawn outside the border of an element.
- An outline is a line drawn around items OUT of boundaries to make the item "stand out".
- Note: the outline is different from the frames! Unlike a border, the outline is drawn outside the element's border and **can overlap** with other content. In addition, the contour is **NOT part** of the element dimensions; the **outline width does not affect** the **overall width** and **height** of the element.
- CSS has the following properties for the outline:
 - `outline-style`
 - `outline-color`
 - `outline-width`
 - `outline-offset`
 - `outline`



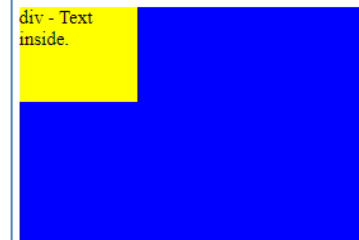
Example – boxes – elementBox (.html/.css)

```
p {
  background-color: bisque;
}
div {
  height: 80px;
  width: 100px;
  background-color: yellow;
}
.outer{
  height: 200px;
  width: 300px;
  background-color: blue;
}
.d1{
  margin: 50px 80px 100px 20px;
  border-color: aquamarine;
  border-width: 5px 15px 20px 30px;
  border-style: solid;
  padding: 30px;
}
```

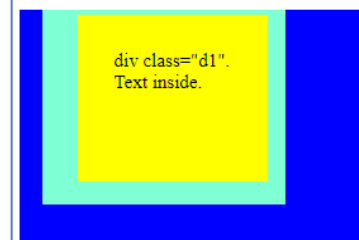
```
<link rel="stylesheet" href="elementBox.css">
<title>CSS3 - Boxes</title>
</head>
<body>
<h1 class="important">CSS3 - Boxes</h1>
<p>anyText</p>
<div class="outer">
  <div>
    div - Text inside.
  </div>
</div>
<p>anyText</p>
<div class="outer">
  <div class="d1">
    div class="d1". Text inside.
  </div>
</div>
<p>anyText</p>
<table border="1">
<tbody>
<tr>
  <td>
    <div class="d1">
      div class="d1". Text inside.
    </div>
  </td>
</tr>
</tbody>
</table>
<p>anyText</p>
</body>
</html>
```

CSS3 - Boxes

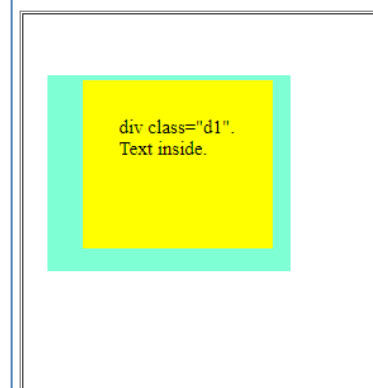
anyText



anyText



anyText



anyText

Example – background – background (.html/.css) 1/2

```
<link rel="stylesheet" href="background.css">
<title>CSS3 - background</title>
</head>
<body>
  <h1 class="important">CSS3 - background</h1>
  <p>anyText</p>
  <div class="outer">
    <div>
      div - Text inside.
    </div>
  </div>
  <p>anyText</p>
  <div class="outer">
    <div class="d1">
      div class="d1". Text inside.
    </div>
  </div>
  <p>anyText</p>
  <table border="1">
    <tbody>
      <tr>
        <td>
          <div class="d1">
            div class="d1". Text inside.
          </div>
        </td>
      </tr>
    </tbody>
  </table>
  <p>anyText</p>
  <p>
    Lorem ipsum dolor sit ...    </p>
</body>
</html>
```

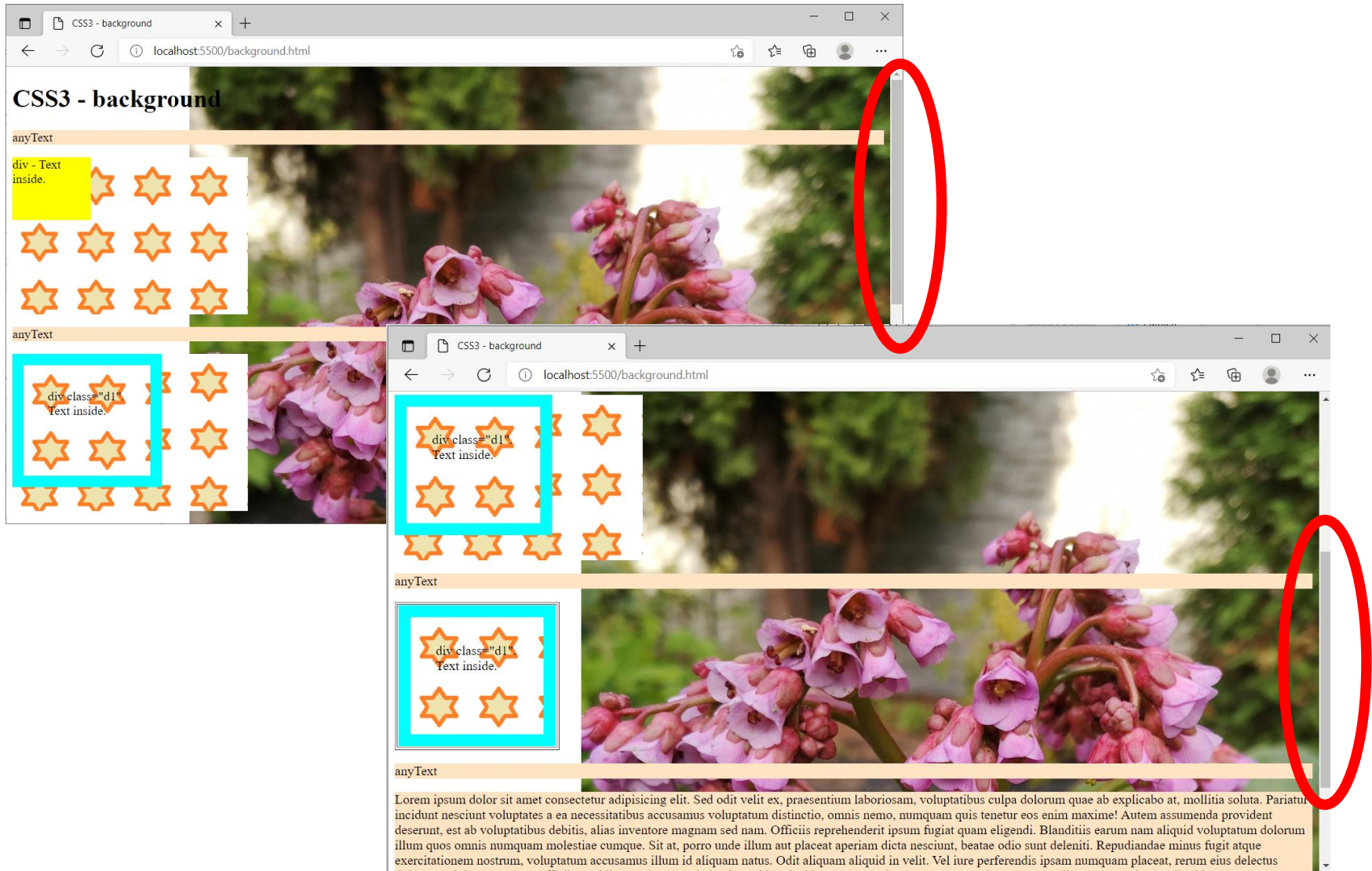
```
body{
  background-image: url("bergenia.jpg ");
  background-repeat: no-repeat;
  background-position: right top;
  background-attachment: fixed;
}
p {
  background-color: bisque;
}
div {
  height: 80px;
  width: 100px;
  background-color: yellow;
}

.outer{
  height: 200px;
  width: 300px;
  background-image: url("star.png");
}

.d1{
  border: 15px solid aqua;

  padding: 30px;
  background-image: url("star.png");
}
```


Example - background - operation 2/2



Relative positioning

- Relative positioning is property **position**=relative
- The element "reserves" the same place as originally occupied, but its image moves relative to its original position (using two properties from: **left**, **right**, **top**, **bottom**).
 - **left** = 15px is the shift from the **left edge** of the starting position by 15px, i.e. the block **shift to the right** by 15px
 - You can use **negative numbers** or opposite properties, ie **left** = 15px is equivalent to **right** = -15px.
 - By default, all of these properties are set to 0.
- Such an element (with **position**=relative) is **required as the parent** of elements that use **absolute** (or **relative**) positioning

An example for a relative position – relative (.html/.css)

```
<link rel="stylesheet" href="relative.css">
<title>CSS3 - position relative</title>
</head>
<body>
  <h1 class="important">CSS3 - position relative</h1>
  <p>Before text <span class="moved">(relative text)</span> after text</p>
  <div id="id1">
    div - id="id1";
  </div>
  <div id="id2" class="moved">
    div - id="id2";
  </div>
  <div id="id3">
    div - id="id3";
  </div>
  <p>anyText</p>
</body>
</html>
```

```
p {
  background-color: bisque;
}
div {
  height: 80px;
  width: 100px;
  background-color: greenyellow;
}
```

```
.moved{
  position: relative;
  background-color: yellow;
  top:10px;
  left:15px;
}
```

```
.moved{
  position: relative;
  background-color: yellow;
  top:10px;
  left:-15px;
}
```

```
.moved{
  position: relative;
  background-color: yellow;
  top:- 10px;
  left:-15px;
}
```

CSS3 - position relative

Before text (relative text) after text

div - id="id1";
div - id="id2";
div - id="id3";

anyText

CSS3 - position relative

Before text (relative text) after text

div - id="id1";
div - id="id2";
div - id="id3";

anyText

CSS3 - position relative

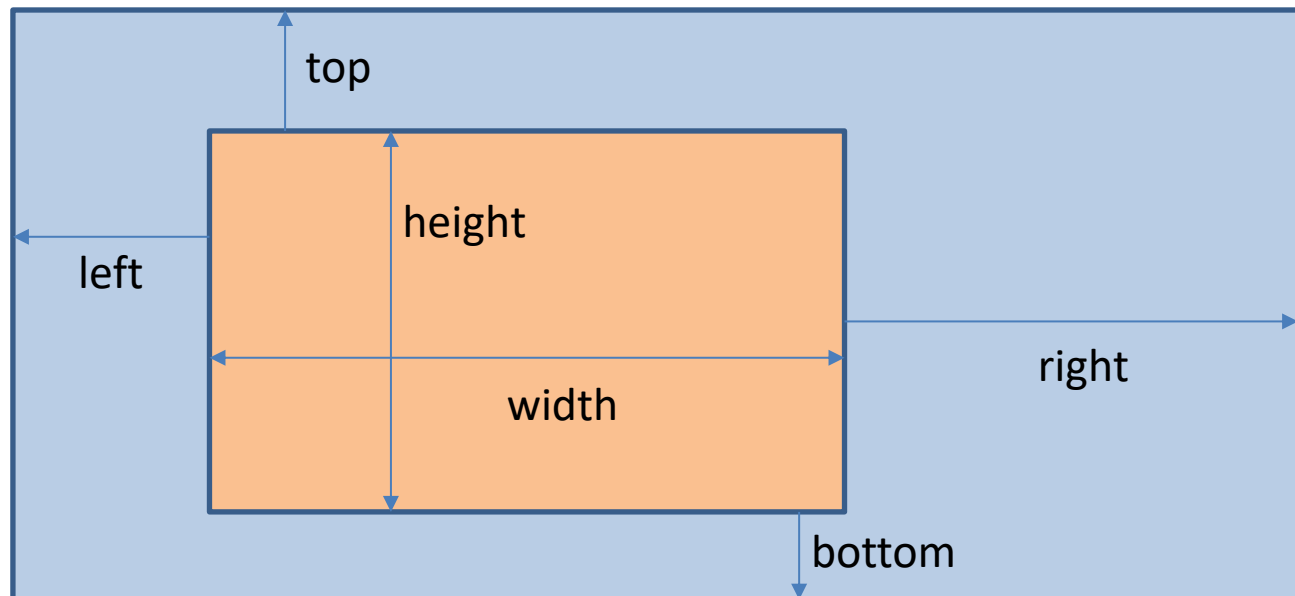
Before text (relative text) after text

div - id="id1";
div - id="id2";
div - id="id3";

anyText

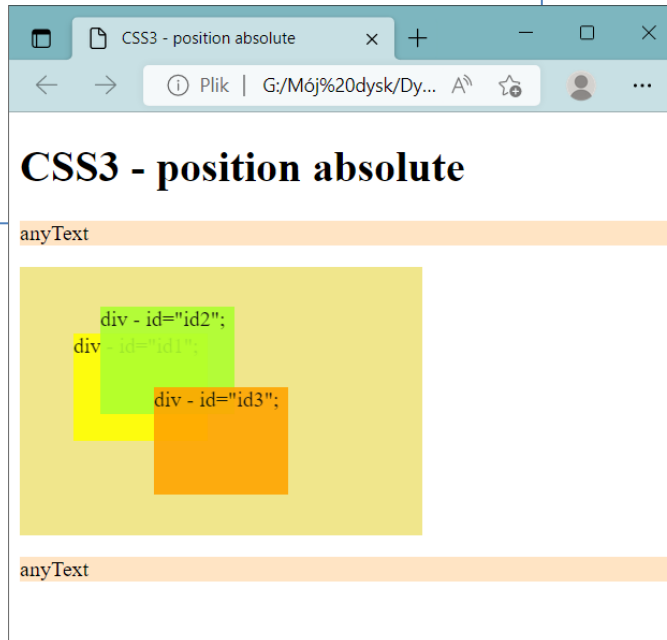
Absolute position of the block element

- Property **position** = `absolute`
 - It stands for absolute position
 - Removes an item from the standard queue of items on the page and places it at the appropriate distance from the top, left, right, or bottom of its ancestor (parent) element.
 - The size of an element can be determined with the CSS properties: **top**, **left**, **bottom**, **right** (the meaning described earlier), **width**, **height**. The set of these properties is redundant, so you should choose 4 of them.
- Property **opacity** = `x%` is the transparency of the element (x = `100` - no transparency, x = `0` - total transparency, **invisible**)



Example - absolute position, absolute (.html/.css)

```
<link rel="stylesheet" href="absolute.css">
<title>CSS3 - position absolute</title>
</head>
<body>
  <h1 class="important">CSS3 - position absolute</h1>
  <p>anyText</p>
  <div class="outer">
    <div id="id1">
      div - id="id1";
    </div>
    <div id="id2">
      div - id="id2";
    </div>
    <div id="id3">
      div - id="id3";
    </div>
  </div>
  <p>anyText</p>
</body>
</html>
```



```
p {
  background-color: bisque;
}
div {
  height: 80px;
  width: 100px;
}
.outer{
  height: 200px;
  width: 300px;
  background-color: khaki;
  position: relative; /* cannot be static */
}
.outer div{
  position: absolute;
  opacity: 90%;
}
#id1{
  background-color: yellow;
  top: 50px;
  left: 40px;
}
#id2{
  background-color: greenyellow;
  top: 30px;
  left: 60px;
}
#id3{
  background-color: orange;
  bottom: 30px;
  right: 100px;
}
```

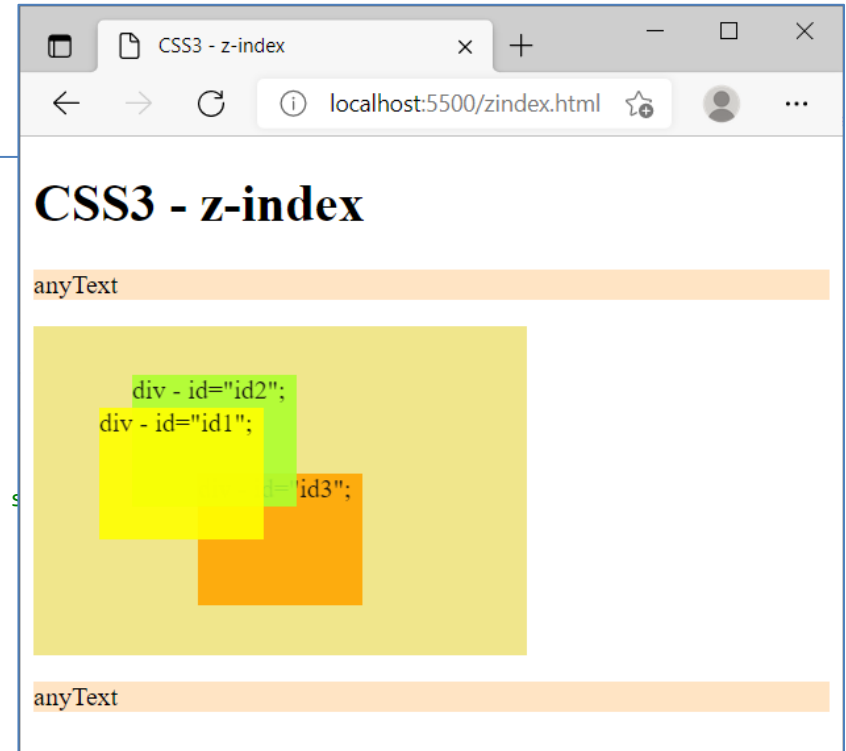
Layers

- In absolute positioning, there is the problem of **covering** the elements. By default, the next element in the HTML text will be drawn over the previous ones.
- The order can be changed by setting the property **z-index** to the appropriate integer.
- An element with a **higher z-index** is drawn **over** the elements at the **lower z-index**.
- By default, all elements have a **z-index** of 0.

An example of absolute and z-index positioning – zindex (.html/.css)

```
<link rel="stylesheet" href="zindex.css"
type="text/css">
<title>CSS3 - z-index</title>
</head>
<body>
<h1 class="important">CSS3 - z-index</h1>
<p>anyText</p>
<div class="outer">
  <div id="id1">
    div - id="id1";
  </div>
  <div id="id2">
    div - id="id2";
  </div>
  <div id="id3">
    div - id="id3";
  </div>
</div>
<p>anyText</p>
</body>
</html>
```

```
p {
  background-color: bisque;
}
div {
  height: 80px;
  width: 100px;
}
.outer{
  height: 200px;
  width: 300px;
  background-color: khaki;
  position: relative; /* cannot be s
}
.outer div{
  position: absolute;
  opacity: 90%;
}
#id1{
  z-index: 2;
  background-color: yellow;
  top: 50px;
  left: 40px;
}
#id2{
  z-index: 1;
  background-color: greenyellow;
  top: 30px;
  left: 60px;
}
#id3{
  background-color: orange;
  bottom: 30px;
  right: 100px;
}
```



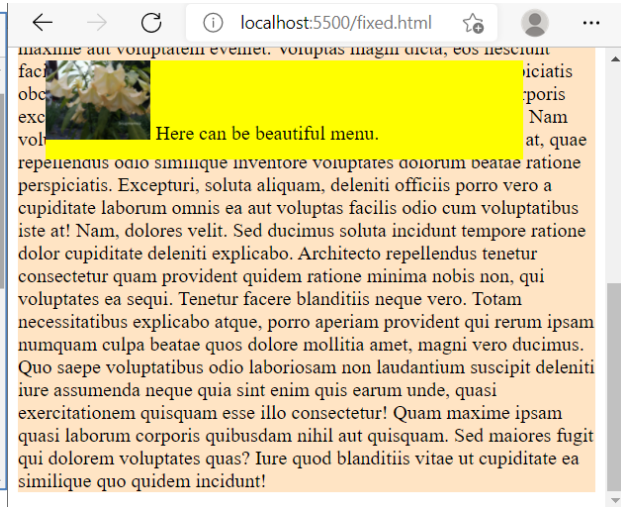
Positioning fixed

- Positioning fixed is property **position**=fixed
 - Like for the background
 - The position of the element is determined with the possible use of 4 (or less) of 6 (**top**, **left**, **bottom**, **right**, **width**, **height**) properties **relative to the viewport** (part of the browser window for the website)
 - Moving the page content with sliders does not move the element
 - Useful for making top / left menu or bottom footer.
However, you must remember that the website should be "empty" in the place of such a menu.

Example - fixed position – fixed (.html/.css)

```
<link rel="stylesheet" href="fixed.css">
<title>CSS3 - position fixed</title>
</head>
<body>
  <h1>CSS3 - obscured title</h1>
  <div id="menu">
    
    Here can be beautiful menu.
  </div>
  <h1>CSS3 - position fixed</h1>
  <p>
    Lorem ipsum dolor sit ...
  </p>
</body>
</html>
```

```
p {
  background-color: bisque;
}
#menu img{
  height: 80%;
}
#menu{
  position: fixed;
  background-color: yellow;
  top:10px;
  left:30px;
  height: 80px;
  width: 80%;
}
```

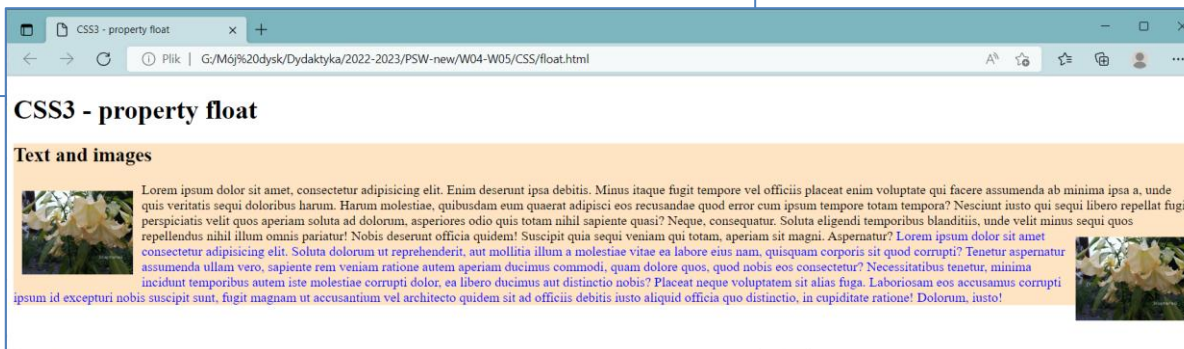


Floating elements

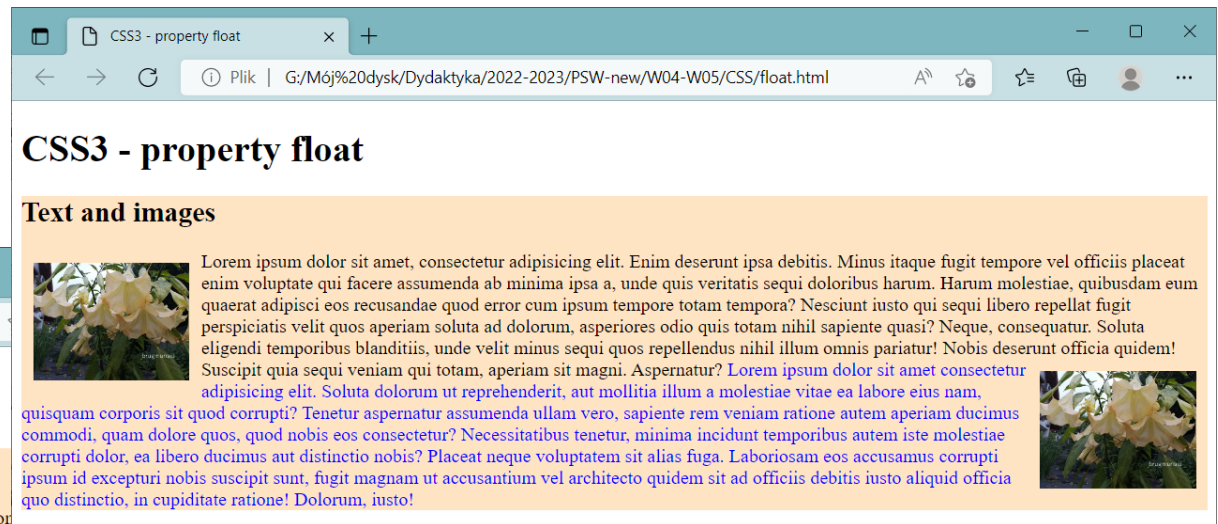
- Property **float** allows you to move an element to the selected edge (**left**, **right**, **top**, **bottom**) of the **parent** element, and the remaining elements will **surround this element**.
- Example:
 - float (.html/.css)

```
<link rel="stylesheet" href="float.css" type="text/css">
<title>CSS3 - property float</title>
</head>
<body>
  <h1>CSS3 - property float</h1>
  <section>
    <h2>Text and images</h2>
    
    Lorem ipsum dolor sit ...
    
    <span style="color:blue">
      Lorem ipsum dolor sit ...
    </span>
  </section>
</body>
</html>
```

```
section {
  background-color: bisque;
}
.image-left{
  float:left;
  text-align: center;
  margin:10px;
}
.image-right{
  float:right;;
  text-align: center;
  margin:10px;
}
```



Other window sizes



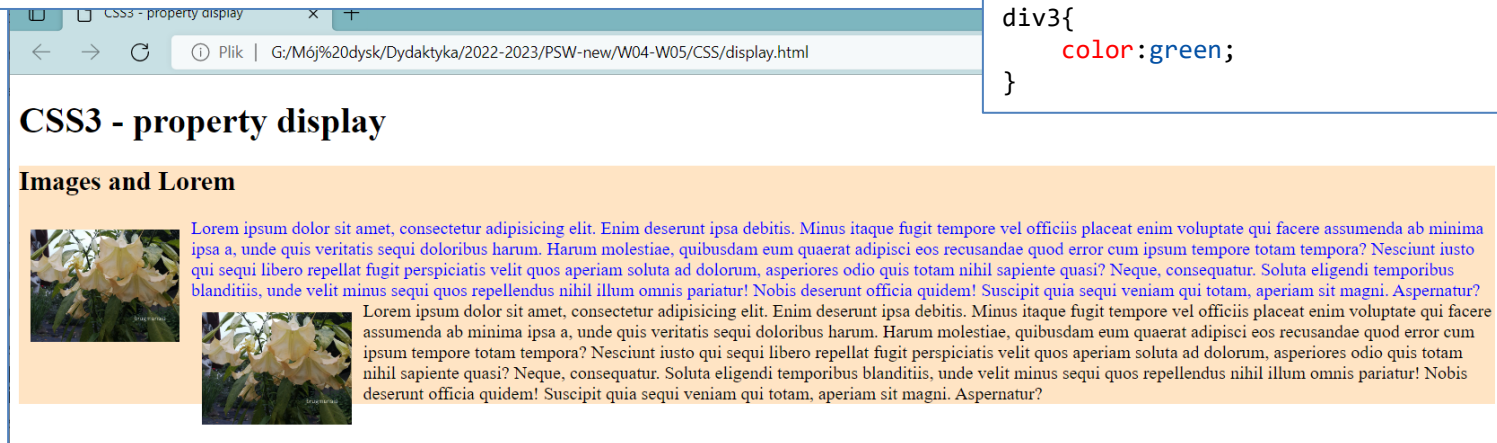
Property **display**

- Property **display** allows the developer to decide whether an item is to be displayed as a block item (`block`), an inline item (`inline`), or not rendered at all (`none`).
 - You can replace the default rendering, which is not advisable, rather you should use the value `none` with the value appropriate for the element.
 - If **display** = `none` then **all children** of that element are also **not visible**
- There are several more values for this property, e.g. `inline-block`, `run-in`, `table` or `table-foot-group`.

An example of display – display (.html/.css) 1/2

```
<link rel="stylesheet" href="display.css" type="text/css">
<title>CSS3 - property display</title>
</head>
<body>
  <h1>CSS3 - property display</h1>
  <section>
    <h2>Images and Lorem</h2>
    <div id="div1">
      
    <div id="div2">
      
    <div id="div3">
      
  </section>
</body>
</html>
```

```
section {
  background-color: bisque;
}
.image-left{
  float:left;
  margin:10px;
}
.image-right{
  float:right;;
  margin:10px;
}
#div1{
  color:blue;
}
#div2{
  display:none; /* display:block/none; */
  color:red;
}
div3{
  color:green;
}
```



An example of display 2/2



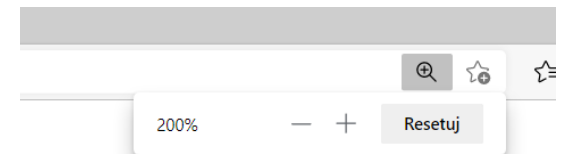
```
section {
    background-color: bisque;
}
.image-left{
    float:left;
    margin:10px;
}
.image-right{
    float:right;;
    margin:10px;
}

#div1{
    color:blue;
}
#div2{
    display:block; /* display:block/none; */
    color:red;
}
div3{
    color:green;
}
```



Conflicting styles

- The style of an HTML element can be described with the help of many selectors:
 - If they are **different properties**, **all** will be **used**
 - If they are the **same style** properties, a **collision** occurs, known as **conflicting** styles.
- Additionally, styles can be defined by the **user agent** (browser) or the **user** himself (in the **browser settings**, but also directly, e.g. by enlarging the page).
- There are general rules:
 - **User-defined** styles **take precedence** over **user agent-defined** styles.
 - **Author-defined** styles **take precedence** over **user-defined** styles.



Style hierarchy

- Most of the styles defined for elements (parents) are also **inherited** by nested elements (children).
 - Hence the word "cascade" in CSS
 - The approach is logical, otherwise if you change e.g. **font-family** in **<body>** you would have to repeat this setting in all internal elements.
- Properties defined **for children** (descendants) have a **higher** so-called **specificity** than the properties defined for **the parents** (ancestors).
- **Conflicts** are resolved in favor of properties with **more specificity**, so the **child's styles** take **precedence**.
- The **inline** style has the **greatest specificity**.
- If the **selector** is **more specific**, it has **more specificity**.
- If **two selectors** are **equivalent**, the **text** to be analyzed **last** has **greater specificity**.
 - Which allows you to use external stylesheets from the selected **framework**, and then add your own stylesheet with style modifications that will "overwrite" those from the framework

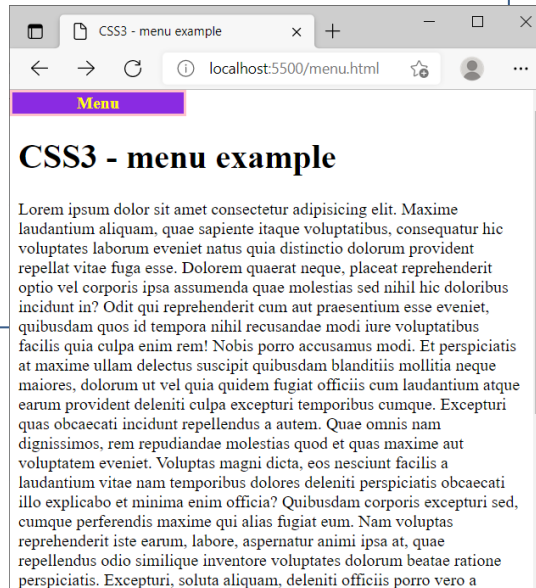
Pseudo-class **hover**

- Pseudo-class **:hover** - it allows you to detect an event consisting in moving the mouse pointer to the area of the selected element.
- The style associated **with** such a **selector** allows for a **dynamic reaction**, e.g. showing hidden elements, etc.
- Example: **creating a menu** from the **<nav>** element that reacts to the position of the mouse over it.
 - The **<nav>** element will be inserted in the upper left corner and will have **fixed** dimensions (and other graphic properties).
 - Apart from the text "Menu" it will contain a list (**** / ****) of links to other pages (**<a>**)
 - The **** internal list for **<nav>** will be hidden (**display: none**), but after hovering the mouse over the text "Menu" (selector **nav: hover ul**) it should appear.
 - The **** elements will have their own style, but their **background color** has to change when the **mouse** is placed **over** one of them (selector **nav ul li: hover**) to make it easier for the user to see what they are about to select.
 - Standard text **decorators** for **<a>** links should be **disabled**.
 - The **top-gap** class will make a **blank space** for the top menu being created.

Example – menu – menu (.html/ .css) 1/2

```
<link rel="stylesheet" href="menu.css" type="text/css">
<title>CSS3 - menu example</title>
</head>
<body>
  <div class="top-gap"></div>
  <h1>CSS3 - menu example</h1>
  <nav> Menu
    <ul>
      <li>
        <a href="fixed.html"> Fixed</a>
      </li>
      <li>
        <a href="elementBox.html"> Element Box</a>
      </li>
      <li>
        <a href="relative.html"> Relative </a>
      </li>
    </ul>
  </nav>

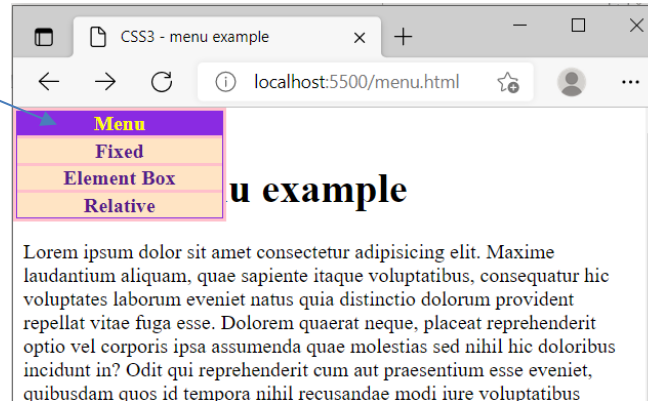
  <p>
    Lorem ipsum dolor
  </p>
</body>
</html>
```



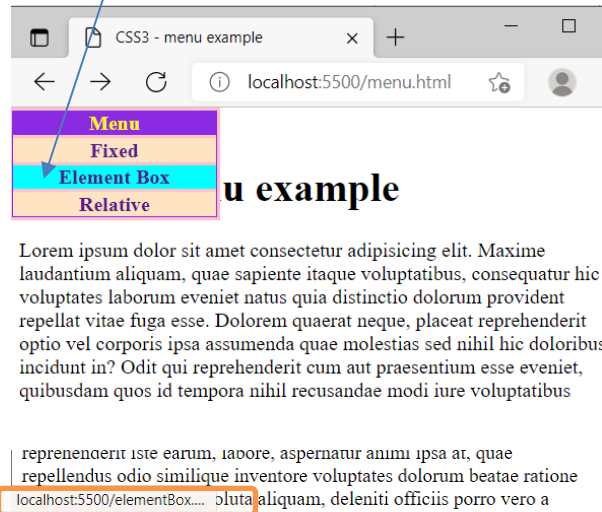
```
.top-gap{
  height: 20px;
}
nav{
  position:fixed;
  left:0px;
  top:0px;
  font-weight: bold;
  color: yellow;
  border: 3px solid pink;
  text-align: center;
  width: 10em;
  background-color:blueviolet;
}
nav ul{
  display: none;
  list-style: none;
  margin: 0;
  padding: 0;
}
nav:hover ul{
  display: block;
}
nav ul li{
  border-top: 3px solid pink;
  background-color:bisque;
  width: 10em;
  color: black;
}
nav ul li:hover{
  background-color: aqua;
}
a{
  text-decoration: none;
}
```

Example – menu 2/2

Mouse pointer



Mouse pointer



When you click the mouse

